Introduction to Machine Learning

Lecture 7: Classification and Logistic Regression

Ali Bereyhi

ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

University of Toronto

Winter 2026

Binary Classification via Linear Models

Let's think of

$$\mathbb{D} = \left\{ \left(\boldsymbol{x}_n \in \mathbb{R}^d, v_n \in \{0, 1\} \right) : n = 1, \ldots, N \right\}$$

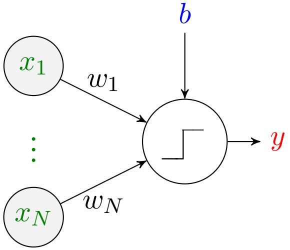
We focus on *linear models*

$$\mathbb{H} = \left\{ f\left(\boldsymbol{x}\right) = \begin{cases} 1 & \mathbf{w}^\mathsf{T} \boldsymbol{x} \geqslant 0 \\ 0 & \mathbf{w}^\mathsf{T} \boldsymbol{x} < 0 \end{cases} \text{for all } \mathbf{w} \in \mathbb{R}^d \right\}$$

Optimal linear model is

$$\mathbf{w}^\star = \underset{\mathbf{w} \in \mathbb{R}^d}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}\left(\mathbf{w}^\mathsf{T} \boldsymbol{x}_n, v_n \right)$$

Linear Classification: *Visualization*



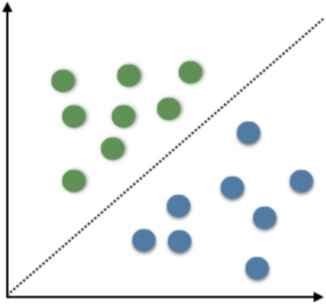
The model is a linear transform followed by a decision-making operation

$$f(\boldsymbol{x}) = \begin{cases} 1 & \mathbf{w}^{\mathsf{T}}\boldsymbol{x} \geqslant 0 \\ 0 & \mathbf{w}^{\mathsf{T}}\boldsymbol{x} < 0 \end{cases} = s\left(\mathbf{w}^{\mathsf{T}}\boldsymbol{x}\right)$$

Linear Classification: *Visualization*

Assume data is in two dimensions

$$f(\boldsymbol{x}) = \begin{cases} 1 & w_1 x_1 + w_2 x_2 \geqslant 0 \\ 0 & w_1 x_1 + w_2 x_2 < 0 \end{cases}$$



Linear Classification: *Empirical Risk*

Typical choice of the loss function

$$\mathcal{L}\left(y_n, v_n\right) = \mathbf{1}\left\{y_n \neq v_n\right\}$$

So the empirical risk is

$$\begin{aligned}\hat{R}\left(\mathbf{w}\right) &= \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}\left\{y_n \neq v_n\right\} \\ &= \textit{Error Rate}\end{aligned}$$

Linear Classification: *Empirical Risk Minimization*

Optimal linear model is

$$\mathbf{w}^\star = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^d} \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}\left\{ s\left(\mathbf{w}^\mathsf{T}\boldsymbol{x}\right) \neq v_n \right\}$$

This is extremely *non-smooth*!

Note

We cannot compute gradient! So, we cannot follow the approach in linear regression

Old Solution: *Perceptron Algorithm*

Start with $\mathbf{w} = \mathbf{0}$ or some small random initial value
while $\hat{R}(\mathbf{w}) \neq 0$ **do**
    for $i = 1 : I$ **do**
        Compute $z_i = \mathbf{w}^\mathsf{T} \boldsymbol{x}_i$ and $\hat{y}_i = s(z_i)$ `# pass through perceptron`
        if $\hat{y}_i \neq y_i$ **then**
            $\mathbf{w} \leftarrow \mathbf{w} - \text{sign}(z_i) \boldsymbol{x}_i$
        end if
    end for
end while

Convergence

You can show that this algorithm converges, if

a line separating the two classes exists

Alternative Solution: *Treating as Regression*

Another solution had been to treat it as a regression!

$$\hat{R}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N}\left(\mathbf{w}^{\mathsf{T}}\boldsymbol{x}_n - \tilde{v}_n\right)^2$$

- If $v_n = 1$: set $\tilde{v}_n = 1$
  - Then, we have $\mathbf{w}^{\mathsf{T}}\boldsymbol{x}_n \approx 1$

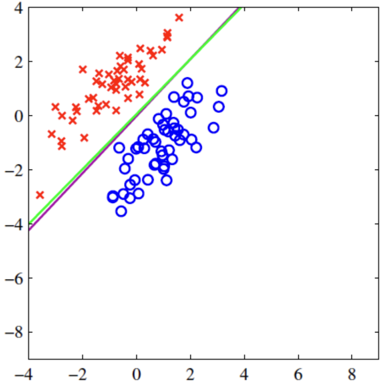
$$f(\boldsymbol{x}) = s\left(\mathbf{w}^{\mathsf{T}}\boldsymbol{x}\right) = 1$$

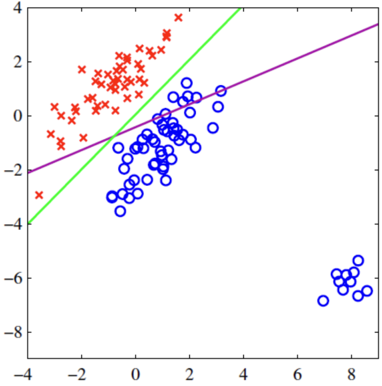
- If $v_n = 0$: set $\tilde{v}_n = -1$
  - Then, we have $\mathbf{w}^{\mathsf{T}}\boldsymbol{x}_n \approx -1$

$$f(\boldsymbol{x}) = s\left(\mathbf{w}^{\mathsf{T}}\boldsymbol{x}\right) = 0$$

Alternative Solution: *Treating as Regression*

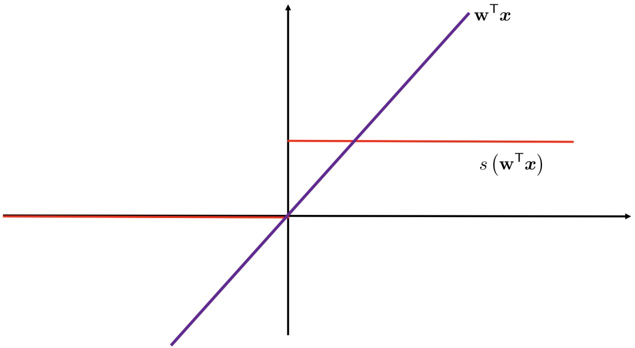
The drawback is though that it is not robust to outliers



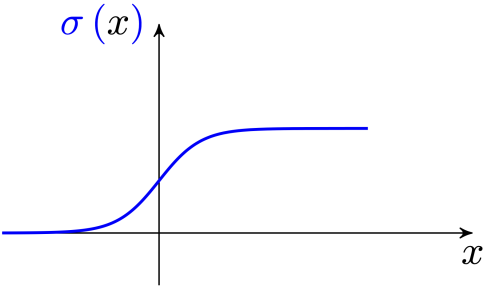


Better Solution: *Thresholding by Sigmoid*

? *Can we do anything in between?*



Better Solution: *Thresholding by Sigmoid*



We can use the sigmoid function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Classification by Sigmoid: *Empirical Risk Minimization*

The empirical risk is then

$$\hat{R}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^{N} \left(\sigma \left(\mathbf{w}^{\mathsf{T}} \boldsymbol{x} \right) - v_n \right)^2$$

We can now compute gradient, and look for

$$\nabla \hat{R} = \mathbf{0}$$

Key Issue

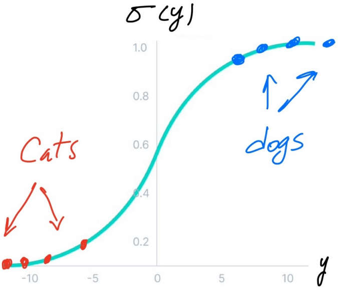
We cannot analytically find it in linear regression!

Logistic Regression: *Soft Information*

For training, we thought of regression problem

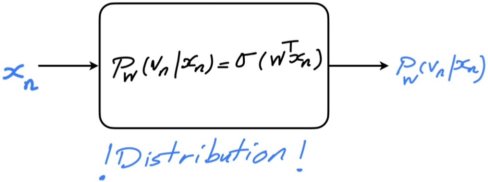
$$\mathbf{w}^\star = \operatorname*{argmin}_{\mathbf{w}\in\mathbb{R}^d} \frac{1}{N}\sum_{n=1}^{N}\left(\sigma\left(\mathbf{w}^\mathsf{T}\boldsymbol{x}\right) - v_n\right)^2$$

? *What are we doing in this approach?*



Logistic Regression: *Sigmoid Output as Probability*

- ! *We know that we compute a probability*



- ! *We are learning a distribution*

$$P_{\mathbf{w}}\left(v|\boldsymbol{x}\right) = \begin{cases} \sigma\left(\mathbf{w}^\top \boldsymbol{x}\right) & v = 1 \\ 1 - \sigma\left(\mathbf{w}^\top \boldsymbol{x}\right) & v = 0 \end{cases}$$

↳ *We could maximize the likelihood*

Logistic Regression: *Maximum Likelihood Estimate*

Let's compute the likelihood of the dataset

$$\mathcal{L}\left(\mathbb{D}\right) = \prod_{n=1}^{N} P_{\mathbf{w}}\left(v_n | \boldsymbol{x}_n\right)$$

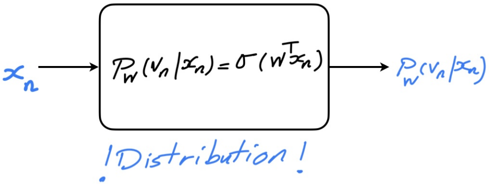
We are more comfortable with log-likelihood

$$\mathcal{S}\left(\mathbb{D}\right) = \frac{1}{N} \sum_{n=1}^{N} \log P_{\mathbf{w}}\left(v_n | \boldsymbol{x}_n\right)$$

Recall: Maximum Likelihood

Find $\mathbf{w}$ *such that the likelihood is maximized*

Logistic Regression: *Maximum Likelihood Estimate*



With $y_n = \sigma\left(\mathbf{w}^{\mathsf{T}} \boldsymbol{x}_n\right)$, we can write

$$\log P_{\mathbf{w}}\left(v_n | \boldsymbol{x}_n\right) = \begin{cases} \log\left(y_n\right) & v = 1 \\ \log\left(1 - y_n\right) & v = 0 \end{cases}$$

$$= v_n \log y_n + \left(1 - v_n\right) \log\left(1 - y_n\right) = \mathcal{S}_n$$

Log Likelihood versus Empirical Risk

Let's now find the maximum likelihood estimate for $\mathbf{w}$

$$
\begin{aligned}
\mathbf{w}^\star &= \operatorname*{argmax}_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \log P_{\mathbf{w}} \left(v_n | \boldsymbol{x}_n \right) \\
&= \operatorname*{argmax}_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^{N} \mathcal{S}_n \\
&= \operatorname*{argmin}_{\mathbf{w}} - \frac{1}{N} \sum_{n=1}^{N} \mathcal{S}_n
\end{aligned}
$$

This is an empirical risk minimization with

$$
\mathcal{L}\left(y, v\right) = -\log P\left(v | \boldsymbol{x}\right) = v \log \frac{1}{y} + \left(1 - v\right) \log \frac{1}{1 - y}
$$

Sample Log Likelihood $\equiv$ Cross Entropy

This has a specific name: *cross-entropy*

Binary Cross-Entropy

Cross-entropy between binary distributions

$$P(v) = \begin{cases} p & v = 1 \\ 1 - p & v = 0 \end{cases} \qquad Q(v) = \begin{cases} q & v = 1 \\ 1 - q & v = 0 \end{cases}$$

is defined as

$$\mathrm{CE}(p, q) = -q \log p - (1 - q) \log (1 - p)$$

Cross Entropy: *Binary Case*

In binary classification, we have

$$\mathcal{L}\left(y, v\right) = -v \log y - \left(1 - v\right) \log \left(1 - y\right)$$

Comparing with the definition, we see that

$$\mathcal{L}\left(y, v\right) = \mathrm{CE}\left(y, v\right)$$

Note

Here, we have $v \in \{0, 1\}$, i.e., we have

$$Q\left(v\right) = \begin{cases} 1 & v = 1 \\ 0 & v = 0 \end{cases} \qquad \textit{or} \qquad Q\left(v\right) = \begin{cases} 0 & v = 1 \\ 1 & v = 0 \end{cases}$$

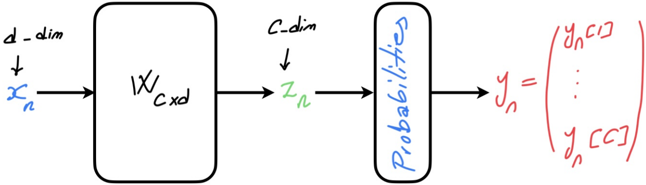
This makes sense, as $Q\left(v\right)$ is distribution of true label that is known to us

Classification with Multiple Classes

? *What if we got more than two classes?*

! *Maybe, we can scale up the idea*

Say we have C different classes $v_n \in \{1, \dots C\}$



We need a function that transforms output of linear model to probabilities

$$\boldsymbol{y}_n = F(\boldsymbol{z}_n) = F(\mathbf{W}\boldsymbol{x}_n)$$

Softmax: *Distribution Out of Soft Information*

Softmax

Softmax transforms a C-dimensional input to a categorical distribution

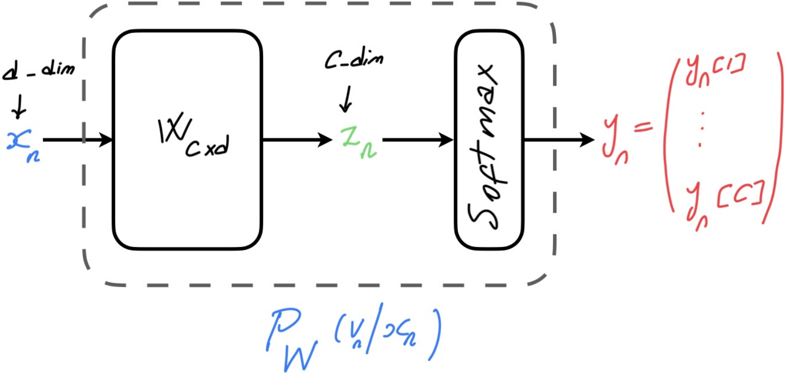
$$y_i = [\text{Soft}_{\max}(\boldsymbol{z})]_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax has the properties we want

- *Its output is a distribution*
- *The output that is larger has higher probabilities*

Maximum Likelihood via *Softmax*

We can again use maximum likelihood to find optimal $\mathbf{W}$



Maximum Likelihood via *Softmax*

We can again use maximum likelihood to find optimal $\mathbf{W}$

$$\mathbf{W}^\star = \operatorname*{argmin}_{\mathbf{W}} \frac{1}{N} \sum_{n=1}^{N} - \log P_{\mathbf{W}} \left(v_n | \boldsymbol{x}_n \right)$$
$$= \operatorname*{argmin}_{\mathbf{W}} \frac{1}{N} \sum_{n=1}^{N} \log \frac{1}{y_n \left[v_n \right]}$$

We can alternatively use one-hot representation of labels

$$\mathbf{u}_n = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{array}{c} \textit{index } 1 \\ \vdots \\ \textit{index } v_n \\ \vdots \\ \textit{index } C \end{array} \rightsquigarrow \log \frac{1}{y_n \left[v_n \right]} = - \sum_{i=1}^{C} u_n[i] \log y_n[i]$$

Cross Entropy: *General Form*

Cross-Entropy

Cross-entropy between two categorical distributions $\mathbf{p}$ *and* $\mathbf{q}$ *is defined as*

$$\mathrm{CE}\left(\mathbf{p}, \mathbf{q}\right) = -\sum_{i=1}^{C} q_i \log p_i$$

So, we could say

$$\mathbf{W}^\star = \operatorname*{argmin}_{\mathbf{W}} \frac{1}{N} \sum_{n=1}^{N} \mathrm{CE}\left(\boldsymbol{y}_n, \mathbf{u}_n\right)$$

We could again say $\boxed{\textit{Maximum Likelihood} \equiv \textit{Sample Mean CE Minimization}}$

Further Read

- Bishop
  - ↳ Chapter 4: *Sections 4.1 – 4.2* ***Linear Classification***
- ESL
  - ↳ Chapter 4: *Section 4.1 – 4.2* ***Linear Classification***